

TECHNICAL DOCUMENTATION

FreelanceHub

A MERN freelance marketplace for posting work, hiring talent, chatting, holding simulated escrow, and completing jobs with reviews.

React 19 + Vite 6

Express + MongoDB

Socket.IO

JWT auth

Repository: github.com/ShadowAN-AB/Freelance_Marketplace

Local app: <http://localhost:5178> · **API:** <http://localhost:5001>

Document date: 21 September 2026 · Version 1.0

Contents

1. 1. What this project is
2. 2. Goals and non-goals
3. 3. Technology stack
4. 4. System architecture
5. 5. Repository layout
6. 6. Domain model
7. 7. Roles and screens
8. 8. Core hiring loop
9. 9. Feature catalogue
10. 10. Client application
11. 11. REST API
12. 12. Real-time chat and presence
13. 13. Security and validation
14. 14. Local setup and demo
15. 15. Operations notes

1. What this project is

FreelanceHub is a full-stack marketplace where **clients** post paid projects and **freelancers** bid with a single proposal. After a hire, the two parties chat, the freelancer submits work (optionally with files), the client can request revisions or approve, and simulated escrow is released. Both sides can then leave a review. An **admin** account can inspect users, projects, reports, and analytics.

The product is a classic MERN app (MongoDB, Express, React, Node.js), not microservices. Payments are simulated: accepting a proposal creates a `Payment` in status `held` ; completing the contract flips it to `released` . There is no Stripe, email, or LLM matching.

One-line summary. Post → propose → hire (escrow hold) → chat → submit work → optional revision → complete → review.

2. Goals and non-goals

In scope

- Email/password auth with JWT and bcrypt
- Project posting, browse, filters, skill matching
- One proposal per freelancer per project
- Hire, contract, simulated escrow
- Deliverable uploads and revision requests
- Project-scoped chat with presence and unread badges
- Saved items, proposal shortlists, reports
- Reviews after completion
- Admin analytics, block users, moderate reports

Out of scope

- Real card payments / Stripe / payouts
- Transactional email or SMS
- Google / social login
- LLM-based matching or cover-letter help
- Milestones or partial releases
- Mobile native apps
- Multi-region deployment

3. Technology stack

Layer	Choice	Why it is used
Client	React 19, Vite 6, Tailwind CSS 4	SPA UI, fast local dev, utility styling
Routing / data	React Router 7, TanStack Query, Axios	Page routes, cached REST fetches
Realtime client	Socket.IO client	Live messages and online dots
Charts	Recharts	Admin analytics
API	Node.js, Express 4	REST + static <code>/uploads</code>
Data	MongoDB 7, Mongoose 8	Documents for users, projects, contracts
Auth	JWT, bcryptjs, express-rate-limit	Stateless login, hashed passwords
Validation	Zod	Request body schemas
Uploads	Multer	Avatars (2 MB images) and deliverables (8 MB)
Realtime server	Socket.IO	Chat rooms + presence map
Hardening	Helmet, CORS, hidden <code>x-powered-by</code>	Baseline HTTP headers

Local ports are chosen to avoid macOS conflicts: the API listens on **5001**, the Vite app on **5178**. Vite proxies `/api` , `/uploads` , `/health` , and `/socket.io` to the API.

4. System architecture

The browser never talks to MongoDB. It talks to Express over HTTP (JSON and multipart) and to the same Node process over WebSockets. JWT is sent as `Authorization: Bearer` for REST and as Socket.IO handshake `auth.token` .

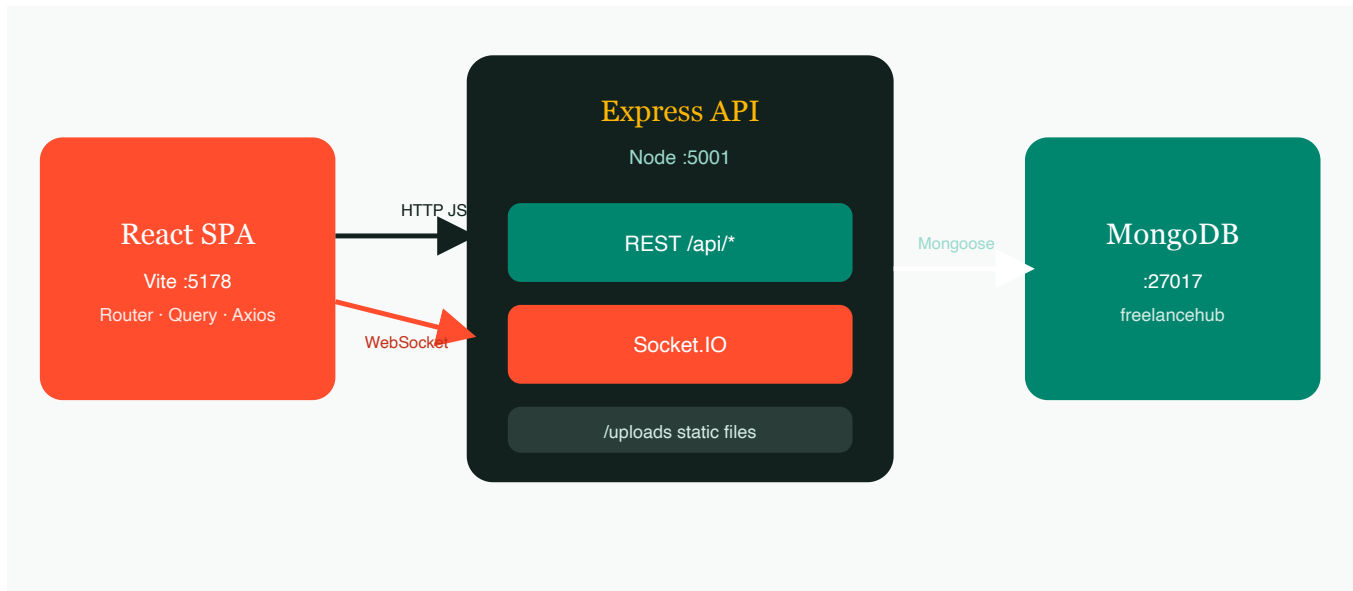


Figure 1. Browser SPA, single Express process (REST + Socket.IO + file hosting), MongoDB.

AuthContext loads `/api/auth/me` from `localStorage.fh_token`, then opens a Socket.IO connection. If the token is missing or rejected, the user is treated as logged out.

5. Repository layout

```

Freelance_Marketplace/
├── client/                React SPA (Vite)
│   ├── src/App.jsx       Routes and role guards
│   ├── src/context/      Auth + socket connect
│   ├── src/layouts/      Public nav and signed-in shell
│   ├── src/pages/        public, auth, client, freelancer, admin, shared
│   └── vite.config.js     Proxies /api and /socket.io → :5001
├── server/
│   ├── src/server.js     HTTP + Socket.IO bootstrap
│   ├── src/models/       Mongoose schemas
│   ├── src/controllers/  Route handlers
│   ├── src/routes/       Express routers
│   ├── src/middleware/   JWT, Zod, Multer, errors
│   ├── src/services/     notify + socket
│   ├── src/seeds/seed.js  Demo data
│   └── uploads/          Avatar and deliverable files
├── docker-compose.yml    mongo:7 on port 27017
├── docs/                 This documentation
└── README.md
  
```

A native `mongod` on the same port as Docker Mongo can hide the empty Docker volume. The API uses whatever is on `MONGO_URI` (default `mongodb://127.0.0.1:27017/freelancehub`).

6. Domain model

Collections are linked by ObjectIds. A project has at most one hired proposal. A contract is unique per project. A payment is unique per contract. Chat is unique per (project, unordered pair of users).

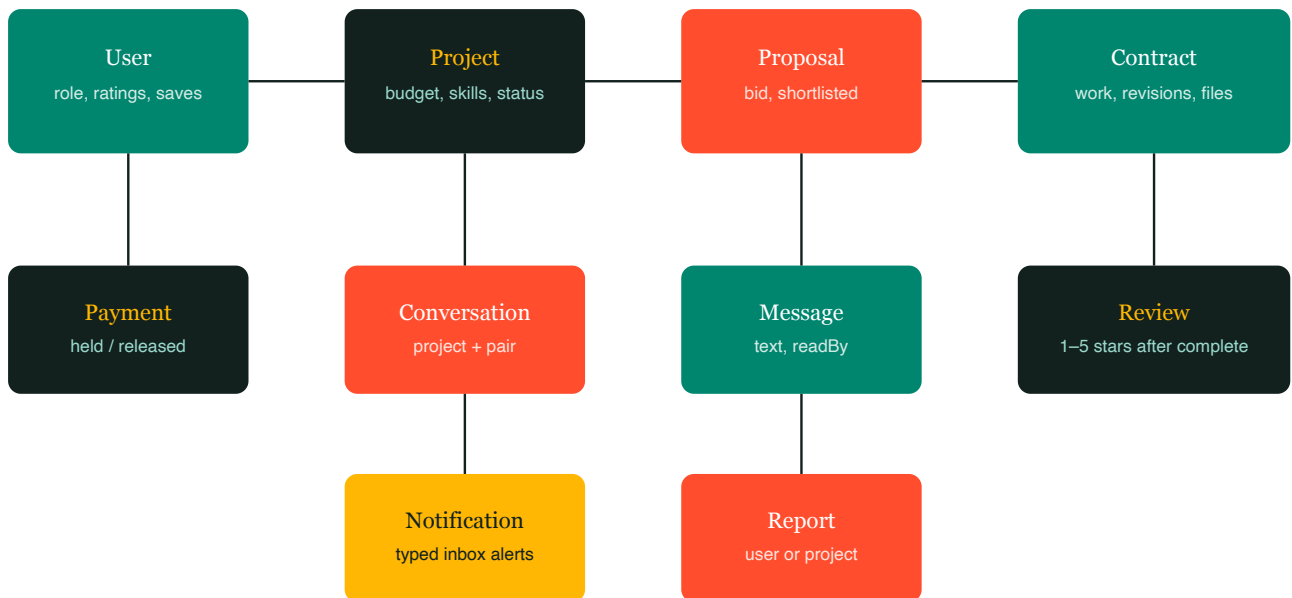


Figure 2. Main collections. Users also hold `savedProjectIds` and `savedFreelancerIds` .

Important statuses

Entity	Statuses
User.role	<code>freelancer</code> , <code>client</code> , <code>admin</code>
Project	<code>open</code> → <code>in_progress</code> → <code>completed</code> (or <code>cancelled</code> / <code>closed</code>)
Proposal	<code>pending</code> , <code>accepted</code> , <code>rejected</code> , <code>withdrawn</code>
Contract	<code>active</code> , <code>completed</code> , <code>cancelled</code>
Payment	<code>held</code> , <code>released</code> , <code>refunded</code>
Report	<code>open</code> , <code>reviewed</code> , <code>dismissed</code>

7. Roles and screens

Role	Can do	Primary screens
Guest	Browse projects and talent. Must log in to bid, save, report, or chat.	Home, /projects, /freelancers, login, register
Freelancer	Propose once per open project, save projects, submit work and files, chat after a proposal exists, withdraw a pending bid, review after completion.	Dashboard, Browse, Proposals, Active work, Saved, Messages, Earnings, Profile, Settings
Client	Post/edit/cancel projects, shortlist and accept/reject bids, save talent, request revision, approve work (release escrow), report, chat after a proposal exists.	Dashboard, Post project, My projects, Proposals inbox, Active work, Saved, Messages, Profile, Settings
Admin	Platform stats, list/block users, list projects, resolve reports. Admin accounts are seeded, not self-registered.	/admin, Users, Projects, Reports

`ProtectedRoute` sends anonymous users to login. `RoleRoute` sends the wrong role back to the dashboard. `GuestRoute` sends an already-signed-in user away from login/register.

8. Core hiring loop

This is the only money path in the product. Everything else (saved items, shortlists, reports, presence) sits on top of it.

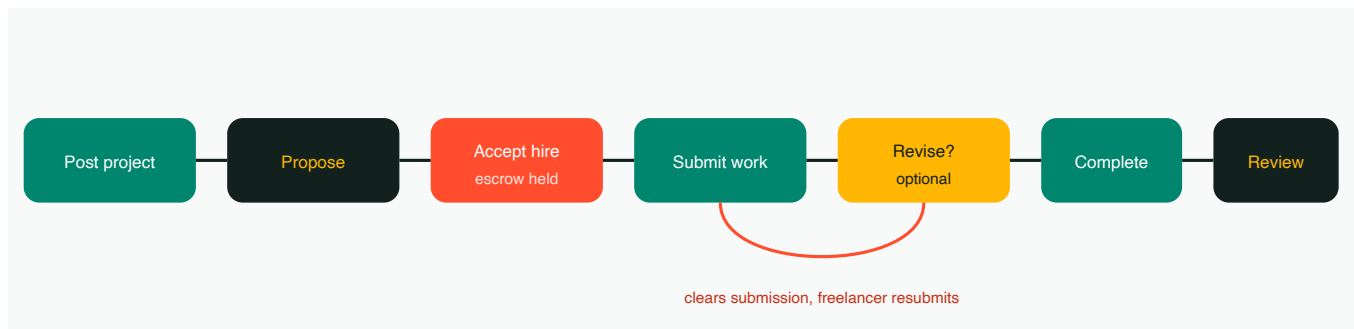


Figure 3. Happy path plus the revision loop. Complete still requires a current `workSubmittedAt`.

Domain rules (enforced on the server)

- One proposal per freelancer per project (unique index).
- You cannot bid on your own project or on a non-open project.
- Accepting a bid rejects other pending bids, sets the project to `in_progress`, creates a contract and a held payment, and opens (or reuses) a conversation.
- Chat is allowed only if a pending or accepted proposal exists between the two users on that project.
- Only the hired freelancer can submit work. Only the client can request a revision or complete.

- Revision keeps status `active` , increments `revisionCount` , stores `revisionNote` , and clears `workSubmittedAt` .
- Reviews are allowed only after `completed` , once per reviewer per contract. Average rating is recomputed on the reviewee.

9. Feature catalogue

9.1 Auth and profiles

Register is limited to `freelancer` or `client` . Passwords are hashed with bcrypt (cost 12). Login is rate-limited (40 attempts / 15 minutes). Blocked users cannot sign in. Profiles store bio, location, avatar, freelancer skills/rate/portfolio/availability, or client company name. `PATCH /api/auth/password` checks the current password before hashing a new one.

9.2 Projects and skill matching

Clients create projects with category, skills, INR budget range, and deadline. Public browse defaults to `open` and supports text search, skill, category, and budget filters. Matching score is overlap of required skills plus a small bonus for extra freelancer skills, capped at 100%. Freelancers see recommended open projects on the dashboard; clients see ranked talent on the proposals page.

9.3 Proposals and shortlist

A proposal has cover letter, bid amount, and estimated days. Clients can star pending proposals (`shortlisted`). The inbox sorts shortlisted first, then newest.

9.4 Work, deliverables, revisions

Submit-work is multipart. Allowed files: PDF, ZIP, PNG, JPEG, WebP, max 8 MB each (up to 8 files). URLs are stored on the contract as `deliverables[]` and served from `/uploads` . Re-submit after a revision replaces the list. The client downloads files from the work card, next to Approve and Request revision.

9.5 Saved items

Freelancers bookmark projects; clients bookmark talent. Toggles live on public detail pages. `/app/saved` lists the populated documents.

9.6 Messaging, unread, presence

Messages persist in MongoDB. Each message has `readBy` . Unread count is the number of threads whose *latest* message is not read by the current user. Opening a thread calls `POST /conversations/:id/read` . Socket.IO maintains `onlineUsers` and emits `presence:update` ; the chat list shows a teal dot when the other participant is connected.

9.7 Notifications

In-app notifications are created for proposal received/accepted/rejected, messages, work submitted, revision requested, contract completed, review received, and report updates. The shell shows a coral count on Alerts.

9.8 Reports and admin

Signed-in users can report a user or a project with a reason. Admins list reports and set status to reviewed or dismissed, which notifies the reporter. Admins can also block users. Analytics aggregate counts by user role, project status, proposal status, contract status, payment status, and reports.

10. Client application

Path	Who	Page
/	Public	Marketing home
/projects , /projects/:id	Public	Browse and project detail (propose, save, report)
/freelancers , /freelancers/:id	Public	Talent directory and profile (save, report)
/login , /register	Guest	Auth forms
/app/dashboard	Signed in	Role-specific desk and recommendations
/app/projects/new , /app/projects	Client	Post and list own projects
/app/projects/:id/proposals	Client	Inbox, shortlist, accept/reject
/app/proposals	Freelancer	Own bids
/app/work	Both	Contracts, files, revision, approve, review
/app/messages	Both	Threads + presence
/app/saved	Both	Bookmarks
/app/earnings	Freelancer	Held vs released totals
/app/profile , /app/settings	Signed in	Profile fields and password
/admin/*	Admin	Stats, users, projects, reports

Visual language: mint paper background, coral primary buttons, teal accents, saffron highlights, Fraunces display type. Nested dashboard routes render through an `Outlet` inside `AppShell` so the sidebar does not unmount the page.

11. REST API

Base URL in the browser is `/api` (proxied). JSON error shape: `{ message, errors? }`.

Method	Path	Auth	Purpose
POST	/auth/register, /auth/login	Public (limited)	Create session JWT
GET	/auth/me	User	Current user
PATCH	/auth/password	User	Change password
PATCH	/users/me	User	Update profile
POST	/users/me/avatar	User	Avatar image
GET/POST/DELETE	/users/me/saved*	Role-specific	Bookmarks
GET	/freelancers	Public	Talent search
GET/POST/PATCH	/projects...	Mixed	CRUD, cancel, matches, recommended
POST/GET	/projects/:id/proposals	Freelancer / client	Create or list bids
POST	/proposals/:id/accept reject withdraw shortlist	Role-specific	Hire flow
GET	/contracts/me, /contracts/:id	Party	Work cards
POST	/contracts/:id/submit-work	Freelancer	Multipart files
POST	/contracts/:id/request-revision	Client	{ note }
POST	/contracts/:id/complete	Client	Release escrow
POST	/contracts/:id/reviews	Party	After complete
GET	/payments/me	Client/freelancer	Escrow ledger
GET/POST	/conversations...	Participant	Threads, messages, read, unread-count
GET/PATCH	/notifications	User	List and mark read
POST	/reports	User	Flag user or project
GET/PATCH	/admin/...	Admin	Stats, users, projects, reports
GET	/health	Public	{ ok, service }

12. Real-time chat and presence

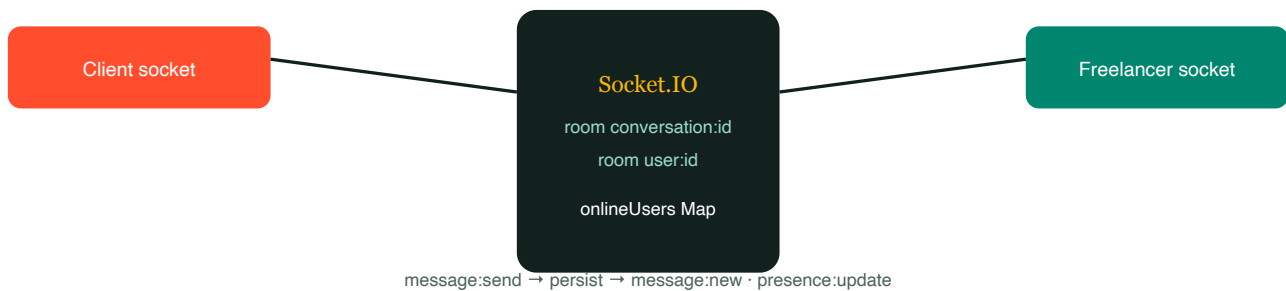


Figure 4. Chat is persisted first, then broadcast. Presence is an in-memory map of `userId` → `socketId`.

Event	Direction	Meaning
<code>conversation:join</code>	Client → server	Join room for live messages
<code>message:send</code>	Client → server	Create message, notify other user
<code>message:new</code>	Server → room	Append in the thread UI
<code>message:read</code>	Server → room	Read receipts
<code>presence:update</code>	Server → all	<code>{ userId, online }</code> ; snapshot sent on connect
<code>notification:new</code>	Server → user room	Nudge alerts after a chat message

HTTP `POST /conversations/:id/messages` is a fallback if the socket is down; it also emits `message:new` when `io` is attached to the Express app.

13. Security and validation

- Passwords stored with `bcrypt`; never returned on user documents (`select: false`).
- JWT payload is `{ id, role }`. Middleware reloads the user and rejects blocked accounts.
- Zod schemas on mutating routes. Multer rejects disallowed MIME types.
- Helmet with cross-origin resource policy so avatars and deliverables can load from the SPA origin.
- CORS locked to `CLIENT_URL`. JSON body capped at 1 MB.
- Auth endpoints rate-limited. Duplicate emails and duplicate proposals return 409.
- Authorization is checked in controllers (client owns project, freelancer is hired party, conversation participant, and so on). Role middleware is a first gate, not the only gate.

Demo secrets. `server/.env` ships a development JWT secret. Rotate it before any shared or production deploy. Payments are not real money.

14. Local setup and demo

Prerequisites

- Node.js 20+ (tested on 22)
- MongoDB on `127.0.0.1:27017` — either Docker Compose or a local `mongod`

```
docker compose up -d
cp .env.example server/.env
cd server && npm install && npm run seed && npm run dev
cd client && npm install && npm run dev
```

Open <http://localhost:5178>. API health: <http://localhost:5001/health>.

Every seeded account uses password `Password123!` .

Role	Email	Notes
Admin	admin@freelancehub.dev	Analytics and moderation
Client	priya@freelancehub.dev	Open projects + in-progress inventory hire with Aisha
Client	arjun@freelancehub.dev	Completed SEO contract with Leo
Freelancer	aisha@freelancehub.dev	Pending dashboard bid + active mobile contract
Freelancer	kabir@freelancehub.dev, meera@freelancehub.dev, leo@freelancehub.dev	Extra bids and a completed job

Ten-minute walkthrough

1. Log in as Priya. Open My projects → React dashboard → Proposals. Star a bid, then Accept (escrow holds).
2. Log in as Aisha. Open Active work, attach a file, submit. Switch to Priya: download the file, Request revision with a note, or Approve and release payment.
3. After a revision, Aisha submits again; Priya can then complete and both leave reviews.
4. Open Messages on both accounts — online dots appear when both sockets are connected. Unread badges sit on Messages and Alerts.
5. Bookmark a project (freelancer) or a freelancer (client) and confirm `/app/saved` .
6. Report a listing from a public page. Resolve it as admin.

15. Operations notes

- `npm run seed` wipes marketplace collections and rebuilds demo users, projects, proposals, contracts, payments, a chat transcript, reviews, and notifications.

- Uploaded files live on disk under `server/uploads/`. They are not in Git.
- Presence is in-memory: a process restart marks everyone offline until they reconnect.
- If port 27017 already has a local MongoDB, Docker's mapped port will fail or you will see an empty database. Point `MONGO_URI` at the instance that actually contains the seed.
- Vite 6 is pinned because newer Vite/Rolldown builds failed on this machine's native bindings.

Source of truth. This document describes the codebase as of 21 September 2026 (main branch including the sequential feature pack: saved items, revisions, deliverables, shortlists, unread badges, settings/reports, and chat presence).